

Dilute models, not results: Liquid State Machines for shrinking neural network parameter counts

Diego Alonso Brule Galleguillos[†], Diana Cristina Andrade Damian[‡]

Abstract—As development of neural network models reach into every aspect of today’s technological life, the need for smaller models is becoming increasingly clear: Whether it is to adapt to limited hardware resources in older or economically accessible technology, to fit into compact technologies like wearables, or to reduce the environmental impact of the computational resources of heavily used neural network solutions like Large Language Models, the importance of the ability to shrink model sizes while not sacrificing performance is paramount in today’s world. In this investigation we show that Liquid State Machines can successfully shrink otherwise complex models and we set out recommendations for adapting our findings to other problems in machine learning. The problem we worked with is hand gesture classification from sequences of Range-Doppler images from Google’s Soli radar, which was developed with the ambition of incorporating gesture control in Google portable devices, where we achieved high accuracies with a model consisting of only one-quarter of the total parameter count and only 0.05% of the trainable parameter count of the best performing model in other experiments [1]

Index Terms—Self Organizing Maps, Unsupervised Learning, Optimization, Neural Networks, Recurrent Neural Networks.

I. INTRODUCTION

In this investigation we show that a well-designed and adapted liquid state machine (LSM) with spiking neurons as a part of a larger neural network can massively shrink models compared to other solutions in the literature without sacrificing performance. While we perform our experiments on the specific problem of gesture recognition using micro doppler radar data from Google’s Soli radar, we believe these findings can be adapted to a wide range of common machine learning problems, as the high performance of our model on this particular problem shows the liquid state machine’s flexible feature representation abilities as we deal with both spatial and temporal data.

There are two main reasons for why LSMs have such large potential in shrinking model parameter counts:

- 1) LSM reservoirs themselves are set to be constant and not trainable yet still have impressive feature extraction capability. Since they are initialised once under certain parameters, replacing network components with LSM reservoirs can drastically reduce the trainable parameter count in a network. A key metaphor that stood out to me in rationalising why a LSM has these abilities without the need of training is that of a pond [2]: If you throw

an object into a pond, it will create ripples, these ripples will follow physical laws and interfere with each other and bounce around the edges and remain in the liquid slowly decaying until it loses its energy. Different objects will create different patterns, and feature extraction from LSM can be seen as interpreting these different liquid patterns.

- 2) Of course, weights, while not trainable, still need to be stored for each cell of the liquid, and for the interaction between input and the liquid. The number of non-trainable parameters required for a LSM is then simply the number of units times the size of the flattened dimension of the input, which means if a good and compact feature representation can be produced and passed to the LSM, the amount of weights in a wider network associated to the LSM can become significantly smaller.

II. RELATED WORK

Our work took starting points from several different publications in the literature:

- 1) *Interacting with Soli: Exploring Fine-Grained Dynamic Gesture Recognition in the Radio-Frequency spectrum* [3]: This was one of the first investigations into the gesture recognition problem with this dataset. From this work we highlight that a model designed with joint convolutional and recurrent neural networks achieved 87% accuracy with an estimated 3.5 million parameters, of which only 3.5k non-trainable. From this work it is also concluded that convolutional neural networks excel at processing spatial data, but a recurrent neural network element is needed to process temporal information.
- 2) *Radar-Based Hand Gesture Recognition Using Spiking Neural Networks* [1]: From this work we highlight that a model designed with a liquid state machine, which is a reservoir computing technique using spiking neurons, with a classifier readout achieved 98% accuracy with an estimated 1.25 million parameters, of which 500k are trainable. This work is good evidence that liquid state machines are flexible, powerful feature extractors both at a spatial and temporal level.
- 3) *On the Computational power of circuits of spiking neurons* [4]: This work describes in detail the mathematical foundation, and the biological motivation behind the design of a spiking neural network.
- 4) *Leveraging Convolutional Sparse Autoencoders for Robust Movement Classification from Low-Density sEMG*

[†]Department of Mathematics, University of Padova, email: diegoalonso.brulegalleguillos@studenti.unipd.it

[‡]Department of Mathematics, University of Padova, email: dianacristina.andradedamian@studenti.unipd.it

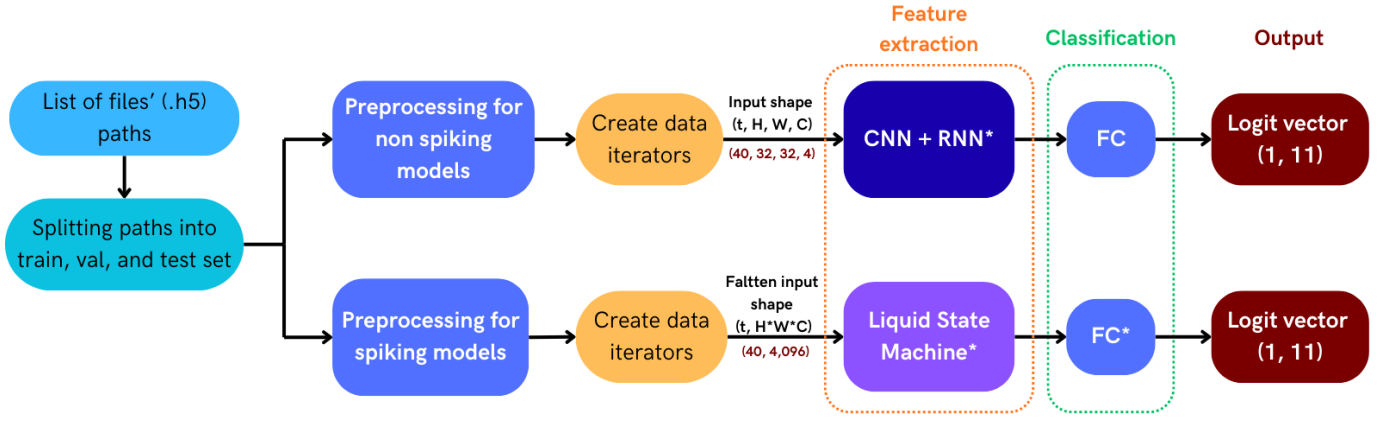


Fig. 1: Processing pipeline's diagram for both spiking and non spiking models. *Structures with multiple variations

[5]: This work uses and trains autoencoders very similarly to our methods.

III. PROCESSING PIPELINE

Data processing, the evolution of inputs as they flow through the models and the interaction between architectural blocks can be seen in *Figure 1*, and is as follows:

- 1) **Preprocessing:** The input data is reshaped and split into training, validation and testing sets. Different additional processing steps, described in detail in the next section, are applied depending on the structure of the input layer of the model. For the convolution-based models we reshape images into RD images, stacked by channel, for each frame of the corresponding radar sequence, as convolution excels at feature mapping spatial data at multiple dimensions. For the LSM-based models, we flatten the images and create binary spike trains, as LSMs output spike trains that encode what neurons were activated and how often, rather than a value of an input evolving while it flows through a reservoir.
- 2) **Input layers:** In the convolution-based models, one set of model variations simply stacks a few convolution layers on top of each other in a simple CNN arrangement, and the other is an encoder arrangement that we pre-trained on the same data. In the CNN model, this spatial feature map is a small-dimensional deep-channel representation, while the encoder outputs a best one-vector representation of the features. In the models with an LSM input layer, we simply feed a flattened RD image into a reservoir.
- 3) **Temporal components:** In our experiments, our temporal-processing components consisted of recurrent neural networks and LSM. Experiments in [3] showed that RNNs on their own are not great at spatial information processing, so we were motivated to place this component after an input layer that would excel at processing this spatial data. LSMs on the other hand are shown [1] to also process spatial data successfully on their own, but using an input layer produced feature

space instead of a full image spike train input into the LSM would result in reducing parameters, as the amount of weights initialised is the size of the LSM flattened input multiplied by the number of units in the reservoir.

- 4) **Classifiers:** for our experiments we applied sequence-wide classifications, rather than frame-by-frame. This comes from our architectural design, as we configure our RNNs and LSMs in such a way that their input has a temporal dimension representing each frame in the sequence, but the output is a one-dimensional feature vector for each sequence. The classification itself was carried out by two methods: The majority of our experiments involved a final fully connected, 11-unit trainable layer that acted as a classifier, and one variation of our models simply used simple classifiers such as logistic regression, random forest or support vector machines

IV. SIGNALS AND FEATURES

A. Input's format

SoliDataset is composed of 5,500 files of h5 type, which is a type of Hierarchical Data Format. The files were obtained by Google's Soli radar, which has 4 antennas, so each file contains 4 channels and the label corresponding to the gesture. Since we are working with gestures, each channel contains a sequence of frames (RDIs) that represents every step of the motion. Frames are composed by 1,024 elements corresponding to the image's pixels, and during the preprocessing phase each file will be reshaped to $(f, 32, 32, 4)$, where f is the number of frames.

It is relevant to mention that file name follows this syntax: `gestureID_sessionID_instanceID`, so gestures, sessions and instances can be accessed through path filtering.

B. Preprocessing techniques

- 1) **Background removal:** We used and slightly adjusted a Gaussian Mixture Model (GMM) [6] originally designed to perform image segmentation in n classes for background removal. We set $n = 2$, which indicates the model to identify two clusters in the data, hand and background.

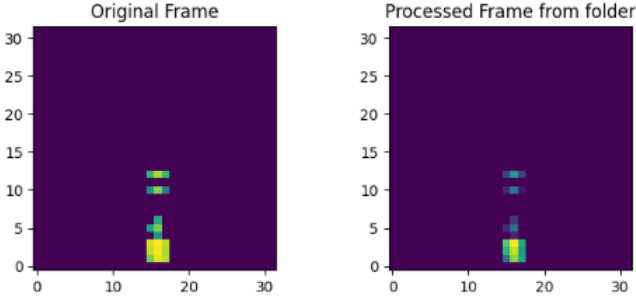


Fig. 2: Example of original RDI before and after applying GMM

Since we are implementing a per-pixel segmentation, applying it to every file's frame would be computationally costly. For this reason, we used this method on the first frame of every sequence and created a background and hand classification mask. Later, background removal is performed by multiplying this mask by the rest of the images, considering that the background pixels correspond to zero. In *Figure 2* we can appreciate the effect of GMM on a RD image.

This procedure was performed on the entire dataset once and saved in a directory in a compressed numpy format that was used to continue the data pipeline.

- 2) **Normalization:** A simple min-max normalization was performed for the data used for non-spiking models, using the following equation,

$$X = \left(\frac{X - \min(X)}{\max(X) - \min(X)} \right). \quad (1)$$

- 3) **Truncation and padding:** Tensorflow data iterators pass the data through batches, so it is very important to unify the number of frames in every sequence. We defined this number to 40, so sequences with more than 40 frames will be truncated, while the ones with fewer will be pad with zeros.

We have three types of truncation; first, middle and random. Most of our experiments were performed using the first 40 frames, but later we included the random mode to improve our data augmentation and exploit our data even more. On the other hand, the padding is always applied to the end of the sequence.

- 4) **Spike generation:** This is a core step to prepare data for spiking models. It basically converts continuous RDI intensity values into a binary signal using the following step function

$$P_i(t) = \begin{cases} 1, & \text{if } f_i(t) > \theta \\ 0, & \text{otherwise} \end{cases}, \quad (2)$$

where $f_i(t)$ is the given signal, and i index the signal at pixel position i , $P_i(t)$ is the output spike state, and θ is a chosen activation threshold. So, our LSM will receive an encoded binary signal as input.

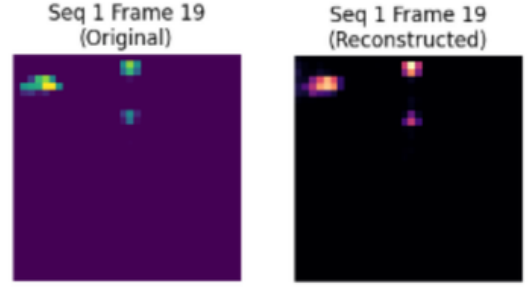


Fig. 3: Visualization of a single frame (RDI) and its reconstructed version from the AE

C. Feature vectors' extraction

For non-spiking models, we used CNN and a simple Autoencoder (AE). CNN accepts an RDI sequence of shape $(f, 32, 32, 4)$ as input. Using three blocks of convolutional and max-pooling layers, it extracts spatial features. The resulting convolved feature maps are then flattened into a 1D feature vector, which serves as the input for the Recurrent Neural Network (RNN).

The AE similarly to CNN performs blocks of convolution to extract the feature maps, which corresponds to the encoding part. The main difference is that it also uses a decoder to reconstruct the original RDI by performing transposed convolution operations. In this way, the AE is trained to learn the most representative 1D feature vector that minimizes the reconstruction error. In *Figure 3* it can be observed an example. However, the decoder is only used during the training phase.

On the other hand, for spiking models we will use a LSM as feature extractor. LSM accepts a flattened RDI sequence of shape $(f, 4096)$, binary encoded. The vector will be multiplied by a fixed random input weight matrix that will distribute the signal from each pixel to multiple neurons within the liquid. These neurons transform the input non linearly following a Leaky Integrate-and-Fire model, where the input spike causes an Excitatory Postsynaptic Potential (EPSP), and when multiple potentials occurred and a certain threshold is reached, a spike is emitted and the neurons' potential is reset, when this occurs we say the neuron fired. On the other hand, the Leaky Integration refers to the potential decay over time that occurs by neuron's inactivity. At each time step t , the state of the reservoir is the vector of all current neurons' activations, and this vector is the actual extracted feature, which will carry spatiotemporal information since the liquid is a recurrent reservoir of neurons.

D. Data splitting:

The data was split in two ways depending on the type of training. For the standard training we shuffle the files' paths and assign 50% for the training set, and 25% for the validation and testing set, respectively.

In addition, we implemented cross-session training to evaluate a personalised classifier. To ensure subject consistency, the dataset was filtered to include only sessions 0, 1, 4, 7, 13, and

14, which were recorded by a single user [3]. In most sessions, the user performed 11 gestures with 50 repetitions each; however, as sessions 13 and 14 contained only 25 repetitions per gesture, they were merged to maintain a balanced distribution across the cross-validation folds. All sessions not belonging to this user were reserved for the independent testing set, which total number of files corresponds to exactly half of the total sequences with 2,750 files.

V. LEARNING FRAMEWORK

A. Architecture's details

1) *Non-spiking models*: Our initial architecture was an End to end (Ete) model composed by a CNN and RNN inspired on the Architecture presented in [3]. The main difference with respect to their Architecture is the presence of pooling layers. The CNN is composed by three convolutional blocks with $kernel = (3, 3)$ and 32, 64 and 128 filters respectively. Also, valid padding is used to preserve the dimensions during the convolution, and max-pooling layers with $kernel = (2, 2)$ are implemented to reduce the feature map by half after every block. The obtained tensor $(40, 4, 4, 128)$ will later be flattened in two consecutive fully connected layers. The output vector will vary depending on the number of neurons set up during the model's configuration. Since CNN excels at capturing spatial features, but not temporal features, it is very important to preserve the order of the sequence during the process, so the RNN can properly extract the temporal features. Finally, the output vector from CNN enters the Long-Short-term memory (LSTM) network that it is later connected to an FC layer with 11 neurons where the classification is represented as a 1D logit vector.

A variant of CNN+RNN was developed by replacing the CNN with an AE, which was trained independently. Later, only the encoder was saved and loaded to create an Ete model by freezing its weights and connecting it to an RNN. Conceptually, CNN and the encoder perform the same convolution operations; however, the decoder plays a key role in the learning process, since it computes a reconstruction error (or loss) that forces the encoder to retain only the most salient features of the RDI frames.

Multiple experiments of these two architecture were tested by modifying the type of pooling layer, adjusting the number of neurons in the FC layers and training with different learning rates. The most relevant findings from these models will be detailed in Section VI.

2) *Spiking models*: : Our initial architectural setup for LSMs was simply to use the LSM to build a table of features for our dataset, and fitting a readout classifier to this, like described in [1]. Since LSMs are initialised randomly once and are not trained, they are highly sensitive to parameters, so we performed an iterative per-dimension search for optimal parameter values. An example of the results of this search can be seen in *Figure 4* and the choice of parameters can be seen in *Table 1*.

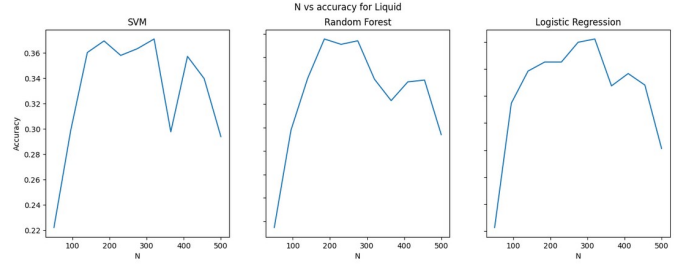


Fig. 4: Number of units vs Accuracy LSM parameter search

TABLE 1: LSM parameter settings.

LSM settings in [1] vs ours				
Units	Leak Rate	Inhib Ratio	Sparsity	C_{inp}
490	0.2	0.2	0.3	0.05
256	0.15	0.35	0.05	1.0

3) *Experimental models*: In addition to the simple LSM into classifier model, we wanted to experiment with adding different components before the LSM in an effort to feed it a more compact feature space and reduce parameter count. Our succesful variations found that adding a CNN or an Encoder before the LSM component yielded equally high accuracy, further proving their spatial processing ability and showing that the LSM can also interpret abstract feature spaces in addition to spatial and temporal information from raw inputs. Since in these two cases the flattened feature map is much smaller than a flattened original image, we were able to maintain model accuracy while reducing parameter counts. We also experimented with adding dense layers between the components, or several dense layers of decreasing unit count between the LSM component and the output logit dense layer, but these did not yield better results, so our best models for these variations are minimalistic.

B. Relevant parameters and Optimization

1) *Pooling layers and padding*: In this work [3], they highlight the negative effect of pooling layers on the accuracy using per-frame prediction. However, without pooling layers, the feature extraction process becomes less efficient and the training process longer. So, in our work pooling layers played one of the most important roles to downsample our feature map, while extracting relevant features. Max Pooling was used in most of the experiments, but also Average Pooling provided promising results.

2) *LSM neuron count*: Like any other neural network, a higher neuron count relates to being able to learn more complex features, while risking overfitting. During our parameter search, we gave preference to finding a significantly smaller neuron count than [1] that would keep an accuracy high.

3) *Leak Rate*: The leak rate of the potential of a Integrate-and-Fire cell. Higher leak rate means that spikes fade faster, giving less priority to the temporal information contained in the input.

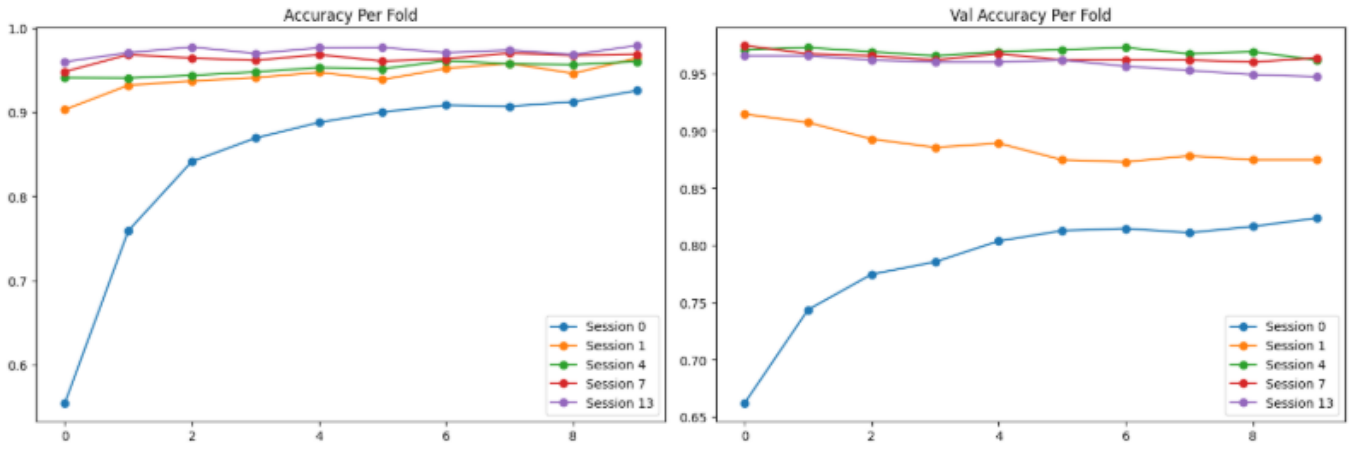


Fig. 5: Training process of model AE+LSM under cross-session validation

4) *Inhibitory Ratio*: This refers to the ratio of cells that are inhibitory, or that emit a negative signal to their forward connections instead of a positive one. This ratio creates a balance in the liquid, too high an inhibitory ratio will quickly fade out any input signal, while too low could allow for chaotic signal behaviour.

5) *Sparsity*: Cells in an LSM reservoir are sparsely connected, and the ratio of this sparsity in relation to full connection tries to find a balance between too-low sparsity not allowing a signal to flow meaningfully while too-high would create chaotic behaviour.

6) *C_{inp} proportion*: These are the amount of connections formed between the input and the reservoir input weights. In theory this can help reduce chaotic behaviour by setting out predefined paths for certain features, but in our parameter search the highest accuracies lied in high values of this parameter or even in full connection.

7) *Learning rate*: We use the following polynomial decay function to gradually reduce our learning rate during the training process and prevent overfitting,

$$lr = lr \times \left(1 - \frac{epoch}{max_epochs}\right)^{0.9}. \quad (3)$$

C. Training and Evaluation process

1) *Standard*: All of our models were trained using this configuration. As mentioned in Section IV, the whole dataset was split into distributing files 2,750 for training and 1,375 for each validation and testing set.

The loss is measured using the Tensorflow Cross-Entropy function, and the evaluation metric is accuracy. During training, we included three relevant callbacks; Gradient Monitor, Polynomial decay function, and Early Stop. Since RNN tends to have a vanishing/exploiting gradient problem, the gradient monitor helps us to identify and prevent these issues by controlling the global norm. The early stop is another tool to prevent overfitting, and it offers the advantage of restoring the best training weights. Additionally, Adam Optimizer was included in all the learning processes.

Finally, most of our models were trained for 50 epochs, with the exception of those where the early stop was triggered. However, those cases with relevant results will be explicitly mentioned.

2) *Leave-one-session-out-cross-validation*: This type of training was only implemented for the best architectures in terms of the complexity-performance trade-off. The main difference from the Standard approach is the data distribution, where 50% of the files form the testing set.

During the training, we have 5-folds where all the files coming from one session remain outside and serve as validation set instead. This process is executed 5 times, ensuring that every session takes the role of validation set once. The number of training epochs is set at 10 per fold, providing a total of 50 for the whole process. In Figure 5 we can observe the learning process of the AE+RNN under a cross-session method.

VI. RESULTS

The whole experimental procedure can be summarized in six different architectures that are described in Table 2. In this section we will describe the reasoning and results that take us to develop each architecture.

A. Complexity-performance trade-off

The initial evaluations performed on the CNN+RNN model provided very good results, with an accuracy value of 0.98. We identified that pooling layers are critical for per-sequence predictions. In one experiment without them, the accuracy dropped to 0.18, and by substituting max pooling layers for average pooling layers, accuracy reached a value of 0.9846. Since we obtained an ideal starting point in terms of accuracy, we decided to focus on reducing the computational costs of our models while preserving the performance.

Following this idea, we explored the effect of an AE as feature extractor. This element was trained independently for only 20 epochs, and the Ete model (AE+RNN) reduced not only the trainable parameters from the feature extractor, but

Layers	CNN+RNN	AE+RNN	LSM+readout	LSM	CNN+LSM	AE+LSM
1	conv1 3*3*32 - pool1	conv1- pool1	LSM 256	LSM 256	conv1- pool1	conv1- pool1
2	conv2 3*3*64 - pool2	conv2- pool2	AvgPool 256	AvgPool256	conv2- pool2	conv2- pool2
3	conv3 3*3*128 - pool3	conv3 - pool3	SVM C=128	FC3 256	conv3 - pool3	conv3 - pool3
4	FC4 512	FC4 128	-	FC4 128	FC4 512	FC4 128
5	FC5 512	FC5 256	-	FC5 64	FC5 512	LSM 256
6	LSTM 512	FC6 256	-	FC6 32	LSM 256	AvgPool256
7	FC7-Logit 11	LSTM 256	-	FC7 16	FC7-Logit 11	FC7-Logit 11
8	-	FC8-Logit 11	-	FC8-Logit 11	-	-
Acc	0.9804	0.9716	0.9654	0.9499	0.9121	0.9345
param*	3,513,643	628,491	0	111,243	97,323	3,339

TABLE 2: **Comparison of network architectures.** All the pooling layers are max pooling with a kernel size of (3, 3), and the average poolings are global operations. *param refers to the number of trainable parameters of each architecture

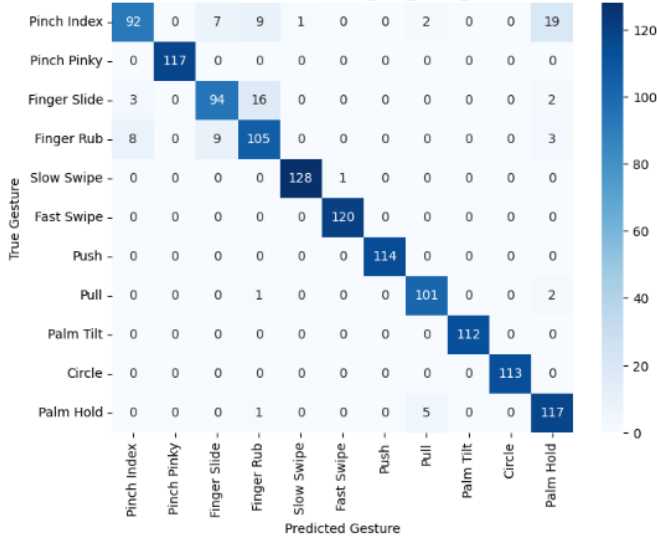


Fig. 6: Confusion matrix of Joint Model AE+LSM under standard training

also in the LSTM by using only half of the initial number of neurons (256), which in consequence decreased the trainable parameters from 3,513,643 (CNN+RNN) to only 628,491, with an accuracy reduction of only 0.01. For this reason, AE+RNN represents a great alternative for non-spiking models.

For the spiking models, the starting point is completely different. The work presented in [1] represents already a solid architecture in terms of complexity-performance trade-off; however, we managed to reduce even more the number of neurons in the Reservoir as shown in the Table 1. In this phase, we noticed that by applying Global Average Pooling after the LSM there was a huge jump in the model's accuracy. The simple LSM model was connected to multiple blocks of FC layers, each of them followed by Batch Normalization and ReLu activation function, leading to a total of 111,243 trainable parameters and 0.95 accuracy score. Again we lost only 0.02 of accuracy compared to the AE+RNN and the parameters were reduced to $\frac{1}{6}$.

Finally, for our last experiments we were already convinced

of the efficacy of LSMs, but we wanted to push the architecture potential to its limits. At this point, we wonder what could happen if we provide as input for the LSM an already smart low dimensional vector. That's how we decided to combine the spatial feature extractors, CNN and AE, to our LSM. For the CNN+LSM the results were not quite convincing, it provided an accuracy of 0.9121 with a small reduction of 14k parameters. On the other hand, the AE+LSM lost 0.015 accuracy compared to the single LSM but it reduced the total number of trainable parameters to only 3,339. This represents only 3% of the total number of trainable parameters from the original LSM, but 0.1% of the CNN+RNN's parameters. In the confusion matrix in Figure 6 the classifications per gesture provided by this model can be found.

From these results, we chose AE+LSM as the best model in terms of complexity-performance trade-off. Provides competitive results while exponentially reducing computational costs. The full description of the number of parameters per layer and memory size is observed in Table 3.

B. Personalized Gesture Classifiers

Finally, this experiment was performed only on two models; AE+RNN and AE+LSM to measure their efficacy as personalized gesture classifiers. The AE+RNN provided an accuracy value of 0.8339, while the AE+LSM obtained a value of 0.7168. It's evident the reduction in accuracy values, but also somehow expected, considering the decrease in the number of training files. More data augmentation and longer trainings are some of the techniques that could have been applied to improve these results.

More research is needed to bring concrete results about these models as personalized gesture classifiers, but temporarily we can say that AE+RNN is still a more robust model for personalized classifiers than AE+LSM.

VII. CONCLUDING REMARKS

To conclude, we want to highlight the main contribution of our work: We propose using LSMs to significantly shrink model parameter counts with respect to known working solutions to different ML problems, and we provided an example of we achieved this, reducing non-trainable parameters and

TABLE 3: AE-LSM Architectural details

Layer (Type)	Output Shape	Param #
inputLayer_1	(None, None, 32, 32, 4)	0
encoder_5d	(None, None, 128)	356,794
lsm_1	(None, None, 256)	98,304
global_average_pooling_1d	(None, 256)	0
batch_normalization	(None, 256)	1024
activation	(None, 256)	0
dense	(None, 11)	2827
Total Parameters	458,859 (1.75 MB)	
Trainable Parameters	3,339(13.34 KB)	
Non-trainable Parameters	455,520 (1.74 MB)	

trainable parameters by one-quarter and 99.5% respectively by applying it to gesture recognition from the Google Soli Radar dataset.

We also want to propose a general approach to adapting LSMs to problems of different contexts: The main focus should be to find the best possible feature extractor relevant to the problem to feed a much more compact yet equally informational input into a LSM. In our case this was by the use of an encoder, which we also trained on this dataset, as autoencoders carry powerful extraction abilities when it comes to image data.

For the LSM itself, we recommend performing thorough parameter searching to find the optimal settings for your problem, as the random nature of a reservoir means our exact settings may not be the optimal settings for a different problem.

REFERENCES

- [1] I. J. Tsang, F. Corradi, M. Sifalakis, W. V. Leekwijck, and S. Latre, "Radar-Based Hand Gesture Recognition Using Spiking Neural Networks," *Electronics*, vol. 10, pp. 851–860, June 2021.
- [2] W. Maas, T. Natschlager, and H. Markram, "Real-Time Computing Without Stable States: A New Framework for Neural Computation Based on Perturbations," *Neural Computation*, vol. 14, pp. 2531 – 2560, Nov. 2002.
- [3] S. Wang, J. Song, J. Lien, I. Poupyrev, and O. Hilliges, "Interacting with Soli: Exploring Fine-Grained Dynamic Gesture Recognition in the Radio-Frequency Spectrum," in *UIST '16: The 29th Annual ACM Symposium on User Interface Software and Technology*, (Tokyo, Japan), Oct. 2016.
- [4] W. Maas and H. Markram, "On the computational power of spiking neurons," *Journal of Computer and System Sciences*, vol. 69, pp. 593–616, July 2004.
- [5] B. Hristov, Z. Hadzi-Velkov, K. H.-V. Saneva, G. Nadzinski, and V. O. Latkoska, "Leveraging convolutional sparse autoencoders for robust movement classification from low-density semg," 2026.
- [6] A. Tripathi, "Gaussian mixture models for background extraction." <https://github.com/akash18tripathi/Gaussian-Mixture-Models-for-Background-Extraction/tree/main>.